



PROGRAM STUDI S1 TEKNIK INFORMATIKA
UNIVERSITAS PAMULANG



Basis Data 2

Pert-13

TRIGGER

VIEW

Tabel virtual yang isinya didefinisikan oleh sebuah kueri (SELECT). View tidak menyimpan data secara fisik dalam cakram keras (kecuali *materialized view*), melainkan mengambil data secara langsung dari tabel asli (*base tables*) setiap kali View tersebut dipanggil.

Fungsi View

- ✓ **Keamanan (Security):** Menyembunyikan kolom atau baris yang sensitif dari pengguna tertentu.
- ✓ **Penyederhanaan (Simplicity):** Menyederhanakan query yang kompleks (**seperti JOIN banyak tabel**) menjadi satu panggilan tabel Tunggal (VIEW)
- ✓ **Konsistensi:** Memastikan semua pengguna melihat format data yang sama tanpa perlu menulis ulang logika query yang rumit.

VIEW pada AdventureWorks2025

Bagian manajemen membutuhkan ringkasan data penjualan produk. Informasi ini tersebar di beberapa tabel seperti **Sales.SalesOrderDetail;** **Production.Product;** dan **Production.ProductSubcategory.** Kita akan menyatukannya ke dalam sebuah View agar staf dapat membacanya dengan satu perintah ringkas

Contoh VIEW pada AdventureWorks2025

```
SQLQuery1.s...nardie (51))* ✕
3      -- Membuat View untuk Ringkasan Penjualan Produk
4      CREATE VIEW Sales.vProductSalesSummary
5      AS
6      SELECT
7          p.ProductID,
8          p.Name AS ProductName,
9          ps.Name AS SubcategoryName,
10         SUM(sod.OrderQty) AS TotalQtySold,
11         SUM(sod.LineTotal) AS TotalRevenue
12     FROM Production.Product p
13     INNER JOIN Sales.SalesOrderDetail sod ON p.ProductID = sod.ProductID
14     LEFT JOIN Production.ProductSubcategory ps ON p.ProductSubcategoryID = ps.ProductSubcategoryID
15     GROUP BY p.ProductID, p.Name, ps.Name;
16     GO
17
```

100 % No issues found Ln: 17, Ch: 1 SP

Messages

CREATE_VIEW - Sales.vProductSalesSummary

Completion time: 2026-05-18T19:41:36.3961172+07:00

-- 2. Menghapus View jika sudah tidak digunakan

```
DROP VIEW Sales.vProductSalesSummary;
GO
```

Memanggil VIEW

SQLQuery2.s...nardie (80))* SQLQuery1.sq...\nardie (51))*

```
1 SELECT * FROM Sales.vProductSalesSummary;
```

100 % 1 0

Results Messages

	ProductID	ProductName
1	714	Long-Sleeve Logo Jersey
2	730	LL Road Frame - Red, 62
3	755	Road-450 Red, 60
4	809	ML Mountain Handlebars
5	795	Road-250 Black, 52
6	782	Mountain-200 Black, 38
7	955	Touring-1000 Yellow, 50
8	867	Women's Mountain Short
9	769	Road-650 Black, 48
10	825	HL Mountain Rear Wheel

SQLQuery2.s...nardie (80))* SQLQuery1.sq...\nardie (51))*

```
1 SELECT TOP 5 *
2 FROM Sales.vProductSalesSummary
3 ORDER BY TotalRevenue DESC;
```

100 % 2 0

Results Messages

	ProductID	ProductName
1	782	Mountain-200 Black, 38
2	783	Mountain-200 Black, 42
3	779	Mountain-200 Silver, 38
4	780	Mountain-200 Silver, 42
5	781	Mountain-200 Silver, 48

SQLQuery2.s...nardie (80))* SQLQuery1.sq...\nardie (51))*

```
1 SELECT *
2 FROM Sales.vProductSalesSummary
3 WHERE SubcategoryName = 'Mountain Bikes';
```

100 % 2 0

Results Messages

	ProductID	ProductName	SubcategoryName	TotalQtySold	TotalRevenue
1	779	Mountain-200 Silver, 38	Mountain Bikes	2394	3693678.025272
2	773	Mountain-100 Silver, 44	Mountain Bikes	601	1217210.359959
3	991	Mountain-500 Black, 44	Mountain Bikes	350	125925.668000
4	785	Mountain-300 Black, 38	Mountain Bikes	684	442477.086952
5	985	Mountain-500 Silver, 42	Mountain Bikes	455	141360.498000
6	782	Mountain-200 Black, 38	Mountain Bikes	2977	4400592.800400
7	776	Mountain-100 Black, 42	Mountain Bikes	634	1254722.732292
8	982	Mountain-400-W Silver, 42	Mountain Bikes	385	217457.874000
9	988	Mountain-500 Silver, 52	Mountain Bikes	515	161293.345200
10	771	Mountain-100 Silver, 38	Mountain Bikes	642	1291868.700375
11	777	Mountain-100 Black, 44	Mountain Bikes	678	1365852.378018

TRIGGER

Trigger adalah blok kode atau prosedur tersimpan (*stored procedure*) yang otomatis berjalan (*fired*) sebagai respons terhadap peristiwa (*event*) tertentu pada tabel. Peristiwa tersebut bisa berupa INSERT, UPDATE, atau DELETE

Fungsi Trigger

- ✓ **Integritas Data:** Memastikan aturan bisnis yang kompleks tetap terjaga (misal: stok barang tidak boleh minus).
- ✓ **Otomatisasi/Log Audit:** Mencatat perubahan data ke tabel log (misal: siapa yang mengubah data dan kapan).
- ✓ **Sinkronisasi:** Memperbarui data di tabel lain secara otomatis ketika terjadi perubahan di tabel utama.

Komponen Utama Trigger

- ✓ **Event (Peristiwa):** Apa yang memicu trigger? Ini biasanya berupa perintah SQL seperti INSERT (tambah data), UPDATE (ubah data), atau DELETE (hapus data).
- ✓ **Condition (Syarat):** Opsional. Apakah ada kualifikasi khusus? (Contoh: Trigger hanya jalan jika stok barang berkurang di bawah angka 10).
- ✓ **Action (Tindakan):** Apa yang harus dilakukan database jika event terjadi? (Contoh: Kirim notifikasi atau catat log aktivitas).

Kapan Menggunakan Trigger

- ✓ **Membuat Log Otomatis:** Mencatat siapa yang menghapus atau mengubah data penting (audit trail).
- ✓ **Integritas Data:** Memastikan jika sebuah data di tabel A dihapus, data yang berkaitan di tabel B juga ikut menyesuaikan.
- ✓ **Kalkulasi Otomatis:** Mengupdate total saldo atau total stok secara otomatis ketika ada transaksi baru.

Implementasi Trigger

Penggunaan **Trigger** pada **SQL Server 2025** menggunakan database sampel **AdventureWorks2025** merupakan cara yang sangat baik untuk mempelajari otomasi database. Secara garis besar, konsep dasar pemicu (trigger) di SQL Server tetap sama, namun pada versi SQL Server 2025, kinerjanya semakin optimal berkat fitur **Optimized Locking** yang meminimalkan overhead penguncian saat trigger berjalan

Implementasi Trigger

Secara umum terdapat dua jenis pemicu DML (Data Manipulation Language) yang paling sering digunakan:

- ✓ **AFTER Trigger (FOR):** Berjalan setelah aksi INSERT, UPDATE, atau DELETE selesai dieksekusi dan lolos validasi constraint
- ✓ **INSTEAD OF Trigger:** Berjalan sebagai pengganti dari aksi DML tersebut (biasanya digunakan pada Views)

Dalam trigger SQL Server menyediakan dua tabel virtual khusus: **inserted** (menyimpan data baru atau data setelah modifikasi) dan **deleted** (menyimpan data lama atau data sebelum dihapus/ diubah)

Contoh Implementasi pada AdventureWorks2025

A. AFTER UPDATE Trigger (Audit Log Modifikasi Data)

Skenario ini digunakan untuk mencatat perubahan ketika ada yang memperbarui data harga produk (StandardCost atau ListPrice) pada tabel Production.Product.

Log perubahan akan disimpan ke dalam tabel audit/ Log Table:

```
SQLQuery2.s...Nardie (76))*
1 CREATE TABLE Production.ProductPriceAudit (
2     AuditID INT IDENTITY(1,1) PRIMARY KEY,
3     ProductID INT,
4     OldListPrice MONEY,
5     NewListPrice MONEY,
6     ModifiedBy NVARCHAR(100) DEFAULT ORIGINAL_LOGIN(),
7     ModifiedDate DATETIME DEFAULT GETDATE()
8 );
```

132 % No issues found

Messages

CREATE_TABLE - Production.ProductPriceAudit

Completion time: 2026-05-18T14:48:49.2339639+07:00

SQLQuery3.s...Nardie (59))* SQLQuery2.sq...Nardie (76))*

```
1 SELECT * FROM Production.ProductPriceAudit;
```

132 % 1 0

Results Messages

AuditID	ProductID	OldListPrice	NewListPrice	ModifiedBy	ModifiedDate
---------	-----------	--------------	--------------	------------	--------------

Membuat Trigger

Trigger yang bertugas otomatis untuk menghubungkan tabel produk utama dengan tabel audit

```
SQLQuery4.s...Nardie (51))*  SQLQuery3.sq...Nardie (59))*  SQLQuery2.sq...Nardie (76))*
1  CREATE TRIGGER TR_Product_UpdatePrice
2  ON Production.Product
3  AFTER UPDATE
4  AS
5  BEGIN
6      SET NOCOUNT ON;
7      -- Memeriksa apakah kolom ListPrice mengalami perubahan
8      IF UPDATE(ListPrice)
9      BEGIN
10         INSERT INTO Production.ProductPriceAudit (ProductID, OldListPrice, NewListPrice)
11         SELECT
12             i.ProductID,
13             d.ListPrice, -- Mengambil data lama dari tabel virtual deleted
14             i.ListPrice  -- Mengambil data baru dari tabel virtual inserted
15         FROM inserted i
16         JOIN deleted d ON i.ProductID = d.ProductID
17         WHERE i.ListPrice <> d.ListPrice; -- Hanya masukkan jika nilainya benar-benar berubah
18     END
19 END;
20 GO
```

120 % 4 0 20

Messages

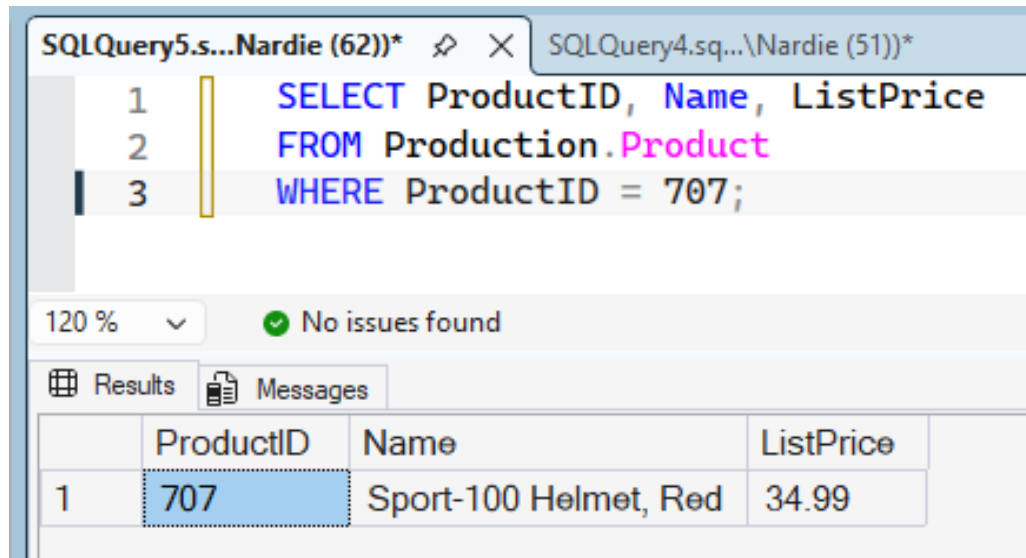
CREATE_TRIGGER - Production.TR_Product_UpdatePrice

Completion time: 2026-05-18T14:55:48.8922182+07:00

Seluruh sistem audit harga otomatis sudah aktif dan siap beraksi!

Membuat Trigger

Cek Data Produk yang Ada:



SQLQuery5.s...Nardie (62))* SQLQuery4.sq...Nardie (51))*

```
1 SELECT ProductID, Name, ListPrice
2 FROM Production.Product
3 WHERE ProductID = 707;
```

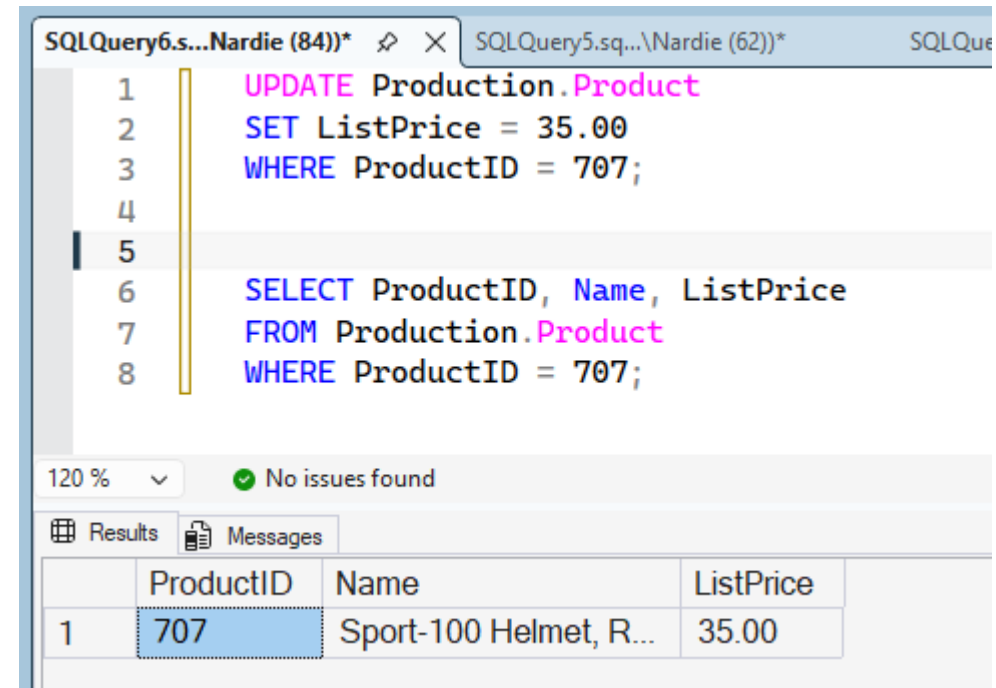
120 % No issues found

Results Messages

	ProductID	Name	ListPrice
1	707	Sport-100 Helmet, Red	34.99

Jika ID 707 tidak ada di database, ganti dengan ID produk lain yang muncul

Lakukan Update Harga (Memicu Trigger)



SQLQuery6.s...Nardie (84))* SQLQuery5.sq...Nardie (62))* SQLQue

```
1 UPDATE Production.Product
2 SET ListPrice = 35.00
3 WHERE ProductID = 707;
4
5
6 SELECT ProductID, Name, ListPrice
7 FROM Production.Product
8 WHERE ProductID = 707;
```

120 % No issues found

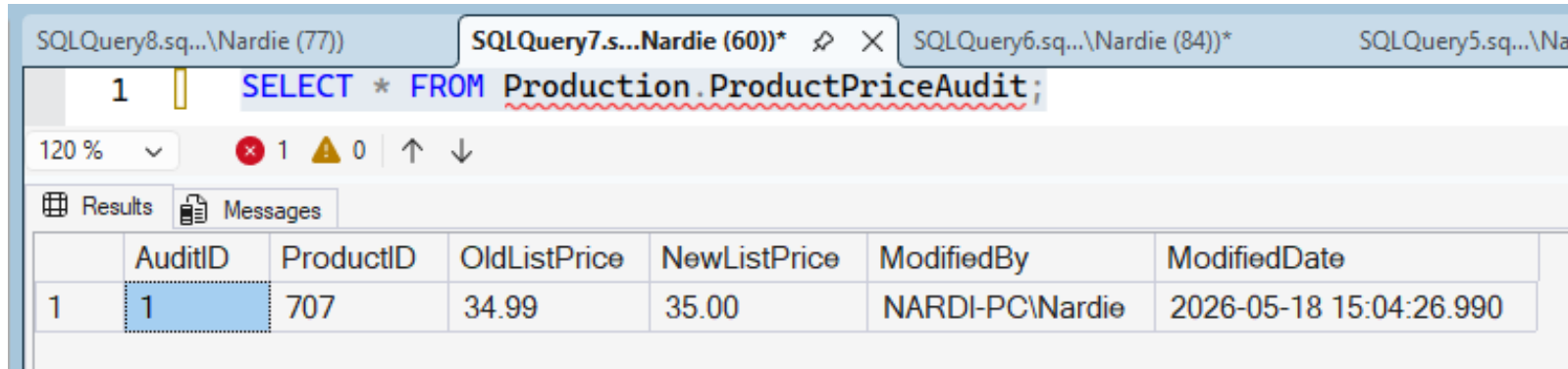
Results Messages

	ProductID	Name	ListPrice
1	707	Sport-100 Helmet, R...	35.00

Begitu perintah Execute, di balik layar Trigger langsung menyergap perubahan tersebut dan mencatatnya ke **tabel audit**

Membuat Trigger

Lihat Hasilnya di Tabel Audit



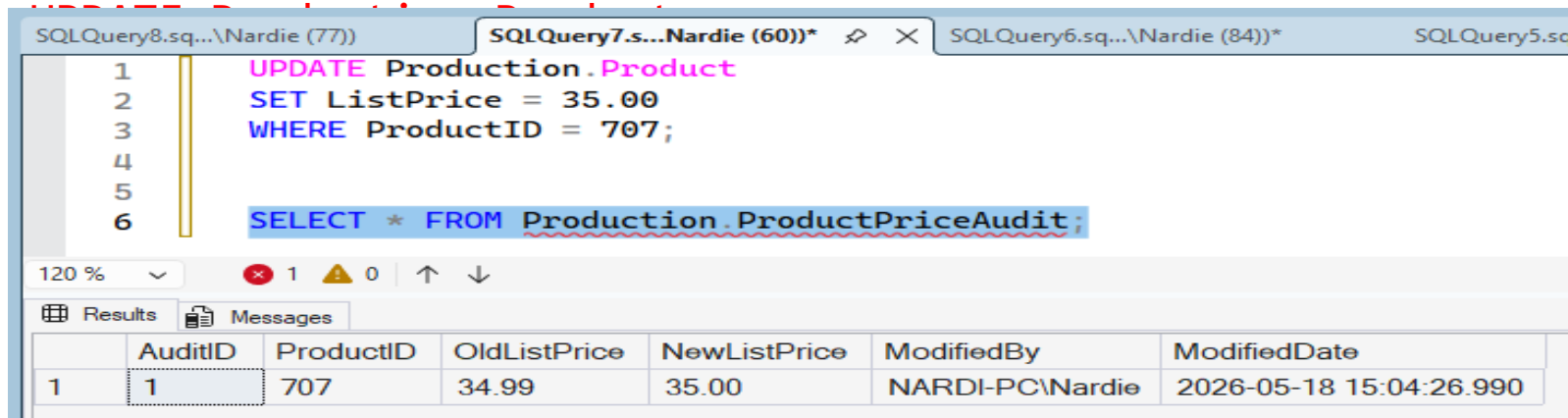
The screenshot shows a SQL query window with the following text: `SELECT * FROM Production.ProductPriceAudit;`. Below the query, the 'Results' tab is active, displaying a table with 7 columns: AuditID, ProductID, OldListPrice, NewListPrice, ModifiedBy, and ModifiedDate. The first row of data is highlighted.

	AuditID	ProductID	OldListPrice	NewListPrice	ModifiedBy	ModifiedDate
1	1	707	34.99	35.00	NARDI-PC\Nardie	2026-05-18 15:04:26.990

Sistem audit database, ada beberapa skenario dunia nyata yang bisa dipelajari atau coba lakukan sekarang

1. Menguji Fitur Keamanan (Mencegah Log Sampah)

Trigger yang di buat memiliki proteksi **WHERE i.ListPrice <> d.ListPrice.**



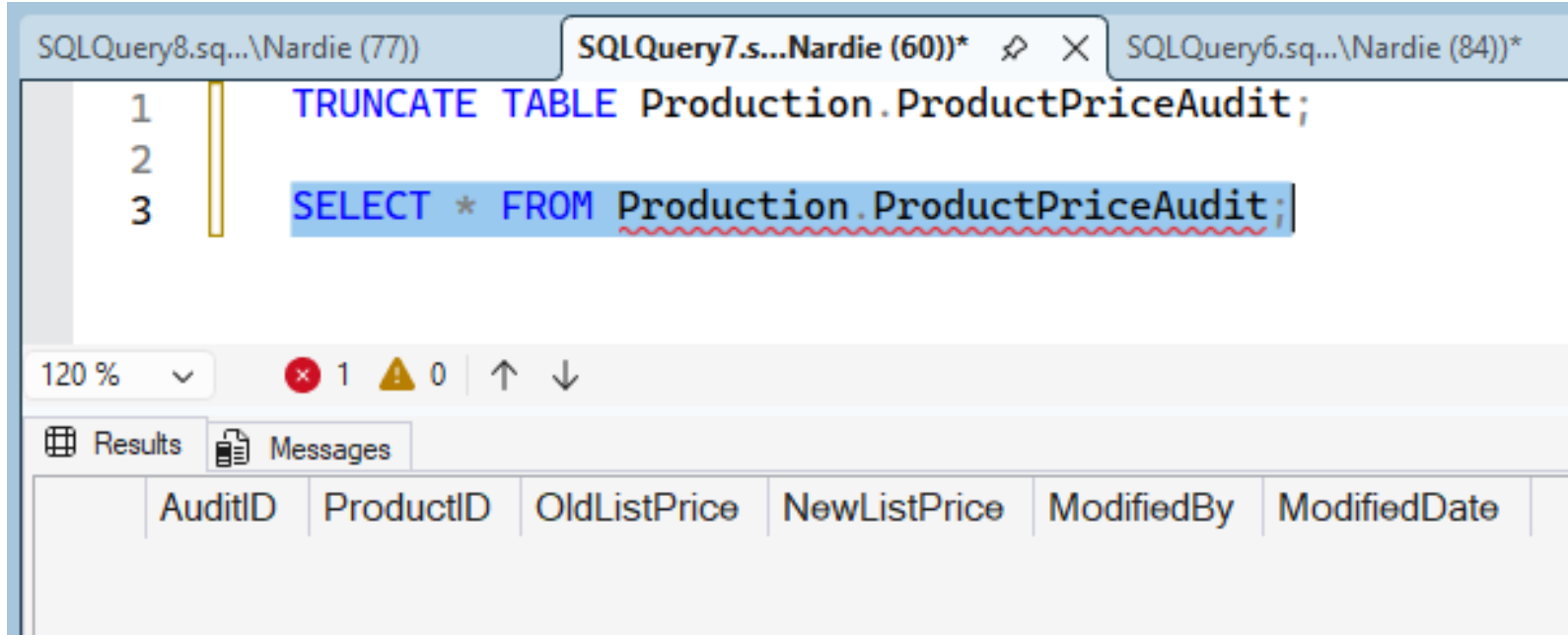
The screenshot shows a SQL query window with the following text: `UPDATE Production.Product SET ListPrice = 35.00 WHERE ProductID = 707;` followed by `SELECT * FROM Production.ProductPriceAudit;`. Below the query, the 'Results' tab is active, displaying a table with 7 columns: AuditID, ProductID, OldListPrice, NewListPrice, ModifiedBy, and ModifiedDate. The first row of data is highlighted.

	AuditID	ProductID	OldListPrice	NewListPrice	ModifiedBy	ModifiedDate
1	1	707	34.99	35.00	NARDI-PC\Nardie	2026-05-18 15:04:26.990

Membuat Trigger

2. Menghapus Log Riwayat (Maintenance)

Tabel audit seperti ini di dunia nyata akan tumbuh menjadi sangat besar seiring berjalannya waktu karena setiap perubahan harga akan terus dicatat. Jika suatu saat ingin membersihkan seluruh riwayat log di tabel audit untuk mengosongkan memori, dapat menggunakan perintah:



The screenshot shows a SQL Server Enterprise Manager interface. At the top, there are three tabs: 'SQLQuery8.sql... \Nardie (77)', 'SQLQuery7.s... \Nardie (60))*', and 'SQLQuery6.sql... \Nardie (84))*'. The active tab is 'SQLQuery7.s... \Nardie (60))*'. The query window contains the following SQL code:

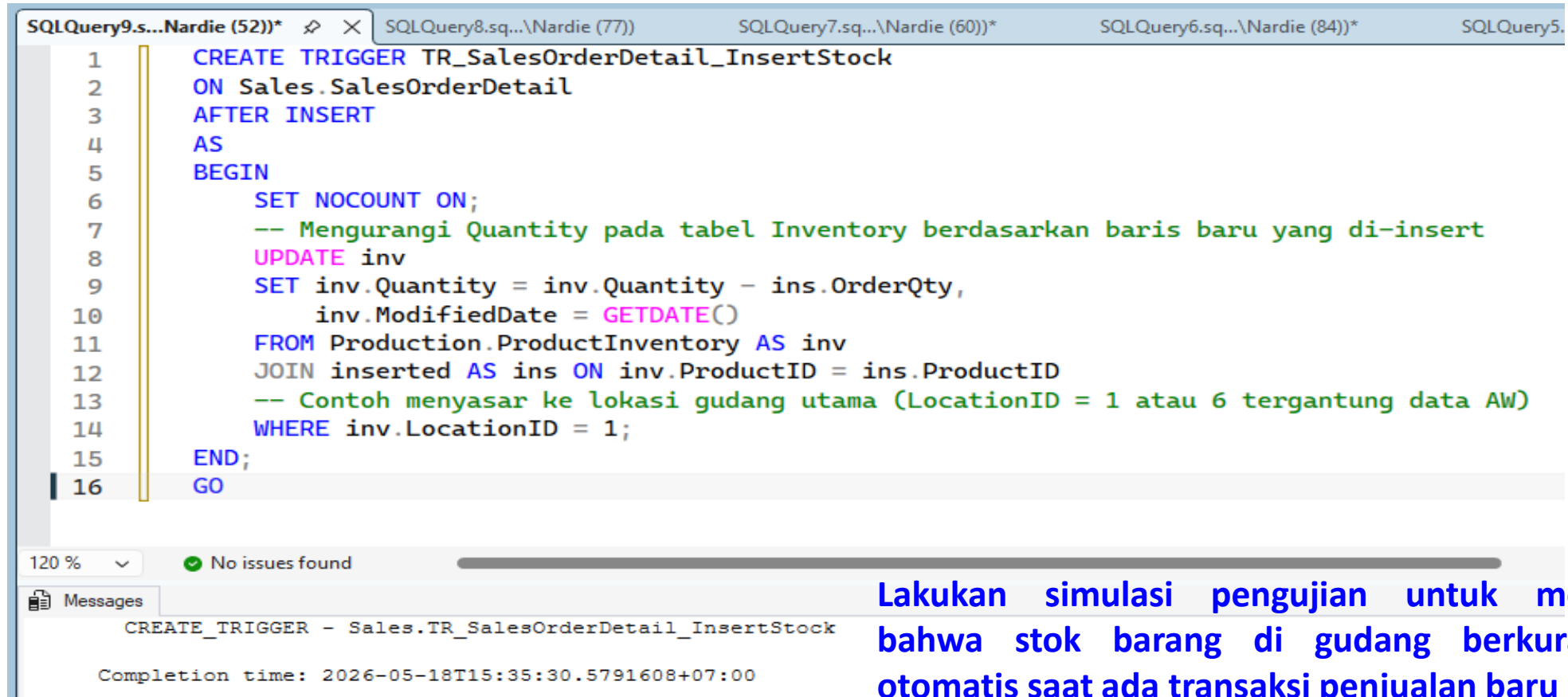
```
1 TRUNCATE TABLE Production.ProductPriceAudit;  
2  
3 SELECT * FROM Production.ProductPriceAudit;
```

Below the query window, there is a status bar showing '120 %' zoom, '1' error, '0' warning, and '0' message. Below the status bar, there are two tabs: 'Results' and 'Messages'. The 'Results' tab is active, showing a table with the following columns: 'AuditID', 'ProductID', 'OldListPrice', 'NewListPrice', 'ModifiedBy', and 'ModifiedDate'.

Contoh Implementasi pada AdventureWorks2025

B. AFTER INSERT Trigger (Otomatisasi Bisnis / Validasi Stok)

Skenario ini digunakan ketika ada transaksi penjualan baru masuk di Sales.SalesOrderDetail, maka jumlah stok barang di gudang (Production.ProductInventory) harus otomatis berkurang sesuai kuantitas yang dibeli



```
SQLQuery9.s...Nardie (52))*  SQLQuery8.sq...\Nardie (77))  SQLQuery7.sq...\Nardie (60))*  SQLQuery6.sq...\Nardie (84))*  SQLQuery5.
1  CREATE TRIGGER TR_SalesOrderDetail_InsertStock
2  ON Sales.SalesOrderDetail
3  AFTER INSERT
4  AS
5  BEGIN
6      SET NOCOUNT ON;
7      -- Mengurangi Quantity pada tabel Inventory berdasarkan baris baru yang di-insert
8      UPDATE inv
9      SET inv.Quantity = inv.Quantity - ins.OrderQty,
10         inv.ModifiedDate = GETDATE()
11      FROM Production.ProductInventory AS inv
12      JOIN inserted AS ins ON inv.ProductID = ins.ProductID
13      -- Contoh menyasar ke lokasi gudang utama (LocationID = 1 atau 6 tergantung data AW)
14      WHERE inv.LocationID = 1;
15  END;
16  GO

120 %  No issues found

Messages
CREATE_TRIGGER - Sales.TR_SalesOrderDetail_InsertStock

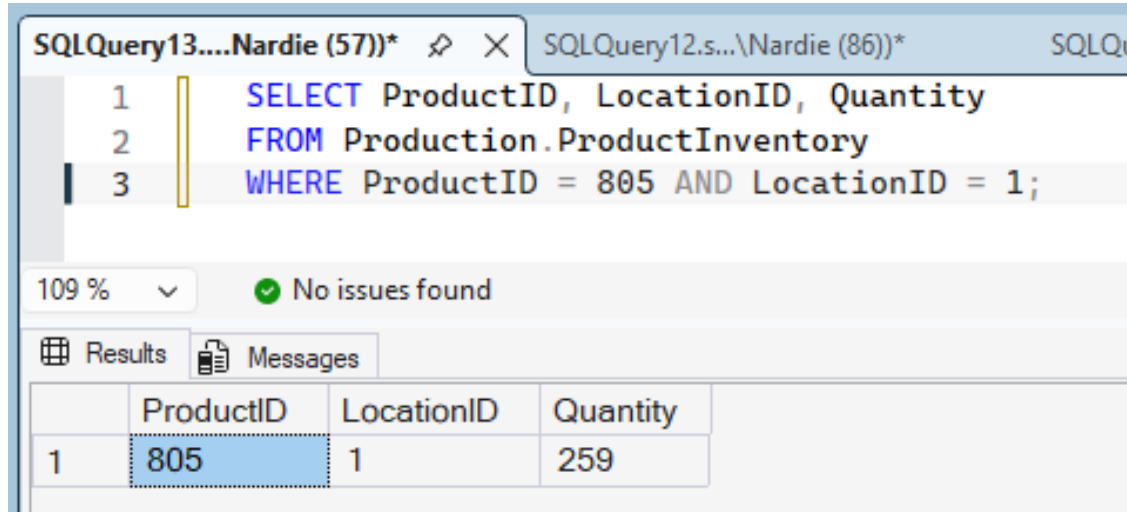
Completion time: 2026-05-18T15:35:30.5791608+07:00
```

Lakukan simulasi pengujian untuk membuktikan bahwa stok barang di gudang berkurang secara otomatis saat ada transaksi penjualan baru masuk!

Contoh Implementasi pada AdventureWorks2025

Cek Stok Awal di Gudang

Cek jumlah stok awal untuk sebuah produk (misalnya produk dengan ProductID = 805) di lokasi gudang utama (LocationID = 1). Jalankan query



The screenshot shows a SQL query window with the following query:

```
1 SELECT ProductID, LocationID, Quantity
2 FROM Production.ProductInventory
3 WHERE ProductID = 805 AND LocationID = 1;
```

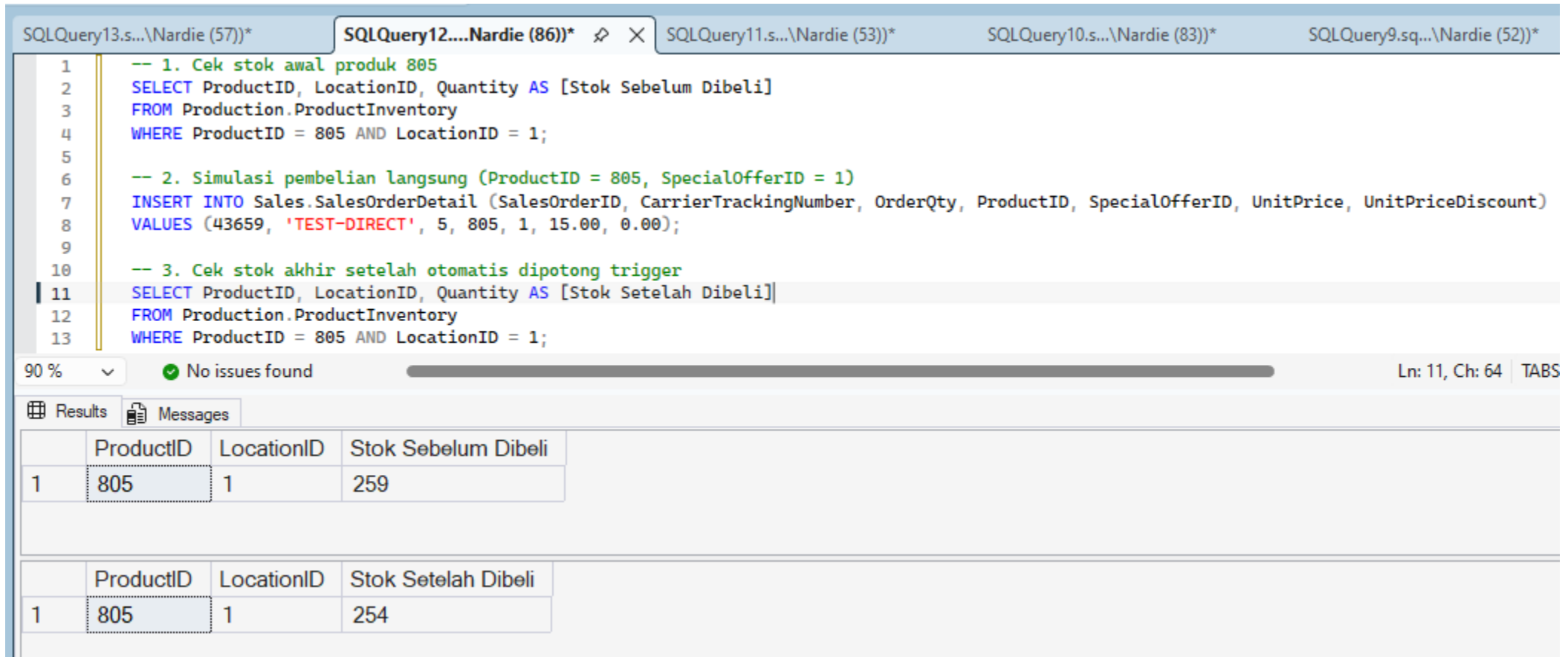
Below the query, the status bar indicates "109 %" and "No issues found". The "Results" tab is active, displaying a table with the following data:

	ProductID	LocationID	Quantity
1	805	1	259

Contoh Implementasi pada AdventureWorks2025

Lakukan Simulasi Transaksi

Misal ada pelanggan membeli produk sebanyak 5 buah. Perlu memasukkan data transaksi baru ke tabel **Sales.SalesOrderDetail**. Sebelumnya butuh satu **SalesOrderID** yang sudah ada di database untuk contoh dan jalankan perintah INSERT:



The screenshot displays a SQL Server query window titled 'SQLQuery12....Nardie (86))*'. The query is divided into three parts:

- 1. Cek stok awal produk 805**: A SELECT statement to check the initial stock of product 805 at location 1. The result shows a quantity of 259.
- 2. Simulasi pembelian langsung (ProductID = 805, SpecialOfferID = 1)**: An INSERT statement into the Sales.SalesOrderDetail table. The values are (43659, 'TEST-DIRECT', 5, 805, 1, 15.00, 0.00).
- 3. Cek stok akhir setelah otomatis dipotong trigger**: A SELECT statement to check the stock after the transaction. The result shows a quantity of 254.

The query window also shows a status bar indicating 'No issues found' and 'Ln: 11, Ch: 64'. Below the query, there are two result tables:

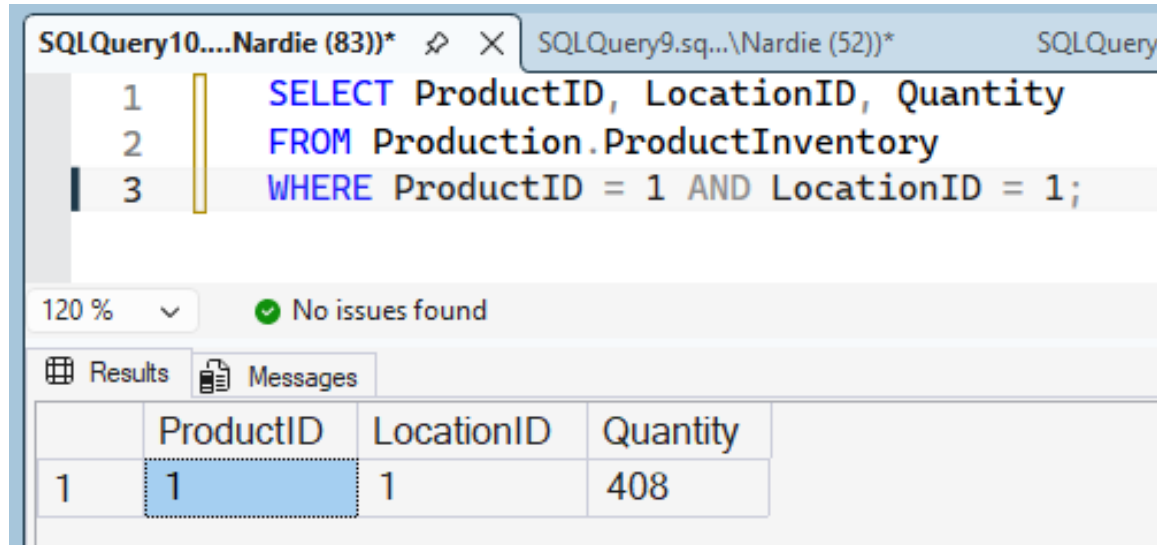
	ProductID	LocationID	Stok Sebelum Dibeli
1	805	1	259

	ProductID	LocationID	Stok Setelah Dibeli
1	805	1	254

Contoh Implementasi pada AdventureWorks2025

Cek Stok Awal di Gudang

Cek jumlah stok awal untuk sebuah produk (misalnya produk dengan ProductID = 1) di lokasi gudang utama (LocationID = 1). Jalankan query



The screenshot shows a SQL query window with the following query:

```
1 SELECT ProductID, LocationID, Quantity
2 FROM Production.ProductInventory
3 WHERE ProductID = 1 AND LocationID = 1;
```

Below the query, the 'Results' tab is active, displaying a table with the following data:

	ProductID	LocationID	Quantity
1	1	1	408

The interface also shows a zoom level of 120% and a status message 'No issues found'.

Contoh Implementasi pada AdventureWorks2025

SQLQuery11....Nardie (53))* SQLQuery10.sq...Nardie (83))* SQLQuery9.sq...Nardie (52))* SQLQuery8.sq...Nardie (77)) SQLQuery7.sq...Nardie (60))*

```
1  -- 1. Cari produk yang VALID dan PASTI ada stoknya di Gudang 1 (LocationID = 1)
2  DECLARE @ValidProductID INT;
3  SELECT TOP 1 @ValidProductID = pinv.ProductID
4  FROM Production.ProductInventory pinv
5  JOIN Sales.SpecialOfferProduct sop ON pinv.ProductID = sop.ProductID
6  WHERE pinv.LocationID = 1 AND pinv.Quantity > 10; -- Pastikan stoknya cukup untuk didebit
7
8  -- 2. Ambil SpecialOfferID yang sah untuk produk tersebut
9  DECLARE @ValidSpecialOfferID INT;
10 SELECT TOP 1 @ValidSpecialOfferID = SpecialOfferID
11 FROM Sales.SpecialOfferProduct
12 WHERE ProductID = @ValidProductID;
13
14 -- 3. CEK STOK AWAL (Akan muncul di tabel atas)
15 SELECT ProductID, LocationID, Quantity AS [Stok Sebelum Dibeli]
16 FROM Production.ProductInventory
17 WHERE ProductID = @ValidProductID AND LocationID = 1;
18
19 -- 4. SIMULASI PEMBELIAN (Beli 5 buah, memicu TRIGGER potongan stok)
20 INSERT INTO Sales.SalesOrderDetail (SalesOrderID, CarrierTrackingNumber, OrderQty, ProductID, SpecialOfferID, UnitPrice, UnitPriceDiscount)
21 VALUES (43659, 'TEST-OK', 5, @ValidProductID, @ValidSpecialOfferID, 15.00, 0.00);
22
23 -- 5. CEK STOK AKHIR (Akan muncul di tabel bawah)
24 SELECT ProductID, LocationID, Quantity AS [Stok Setelah Dibeli]
25 FROM Production.ProductInventory
26 WHERE ProductID = @ValidProductID AND LocationID = 1;
```

90 % No issues found Ln: 12, Ch: 35 TABS

Results Messages

	ProductID	LocationID	Stok Sebelum Dibeli
1	805	1	284

	ProductID	LocationID	Stok Setelah Dibeli
1	805	1	279

Contoh Implementasi pada AdventureWorks2025

3. INSTEAD OF DELETE Trigger (Soft Delete / Pembatasan Penghapusan)

Skenario ini mencegah penghapusan fisik pada data karyawan di **HumanResources.Employee**. Alih-alih menghapus data secara permanen, trigger akan mengubah status karyawan tersebut menjadi tidak aktif (CurrentFlag = 0)

```
CREATE TRIGGER TR_Employee_InsteadOfDelete
ON HumanResources.Employee
INSTEAD OF DELETE
AS
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    -- Melakukan soft delete dengan mengubah flag status, bukan menghapus barisnya
```

```
    UPDATE HumanResources.Employee
```

```
    SET CurrentFlag = 0,
```

```
        ModifiedDate = GETDATE()
```

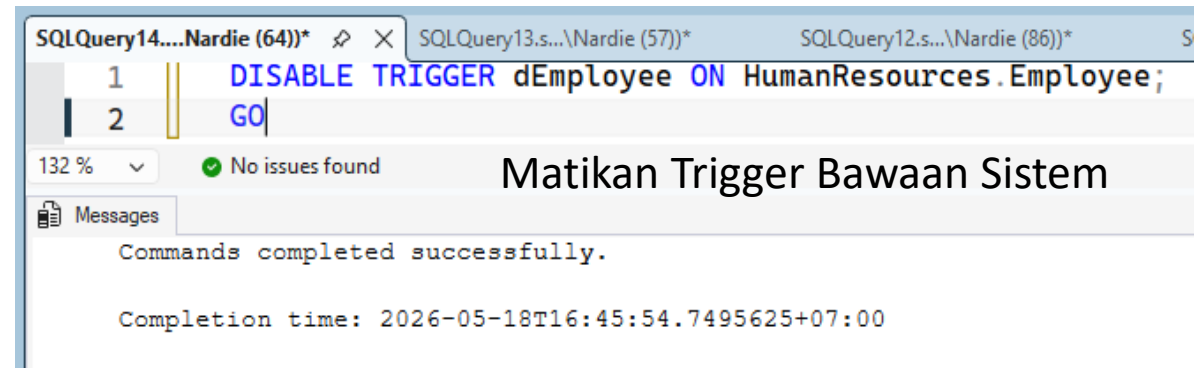
```
    FROM HumanResources.Employee e
```

```
    JOIN deleted d ON e.BusinessEntityID = d.BusinessEntityID;
```

```
    PRINT 'Data tidak dihapus secara fisik, status karyawan telah diubah menjadi Non-Aktif.';
```

```
END;
```

```
GO
```



Contoh Implementasi pada AdventureWorks2025

Buat Table Tiruan

```
SQLQuery16....Nardie (54))*  SQLQuery15.s...\Nardie (93))*  SQLQuery14.s...\Nardie (64))*
1  -- Buat tabel karyawan tiruan
2  CREATE TABLE HumanResources.Employee_Simulasi (
3      BusinessEntityID INT PRIMARY KEY,
4      NamaKaryawan NVARCHAR(100),
5      CurrentFlag BIT DEFAULT 1, -- 1 = Aktif, 0 = Non-Aktif
6      ModifiedDate DATETIME DEFAULT GETDATE()
7  );
8  GO
9
10 -- Masukkan data sampel
11 INSERT INTO HumanResources.Employee_Simulasi (BusinessEntityID, NamaKaryawan)
12 VALUES (1, 'Nardi Test');
13 GO
```

90 % No issues found

Messages

CREATE_TABLE - HumanResources.Employee_Simulasi

(1 row affected)

Completion time: 2026-05-18T16:50:35.4366000+07:00

Contoh Implementasi pada AdventureWorks2025

Buat INSTEAD OF TRIGGER

```
SQLQuery17....Nardie (94))*  SQLQuery16.s...Nardie (54))*  SQLQuery15.s...Nardie (93))*
1  CREATE TRIGGER TR_EmployeeSimulasi_InsteadOfDelete
2  ON HumanResources.Employee_Simulasi
3  INSTEAD OF DELETE
4  AS
5  BEGIN
6      SET NOCOUNT ON;
7
8      -- Mengubah perintah DELETE fisik menjadi UPDATE status (Soft Delete)
9      UPDATE HumanResources.Employee_Simulasi
10     SET CurrentFlag = 0,
11         ModifiedDate = GETDATE()
12     FROM HumanResources.Employee_Simulasi e
13     JOIN deleted d ON e.BusinessEntityID = d.BusinessEntityID;
14
15     PRINT 'Sistem: Data tidak dihapus fisik, status diubah menjadi Non-Aktif!';
16 END;
17 GO
```

90 % 7 0

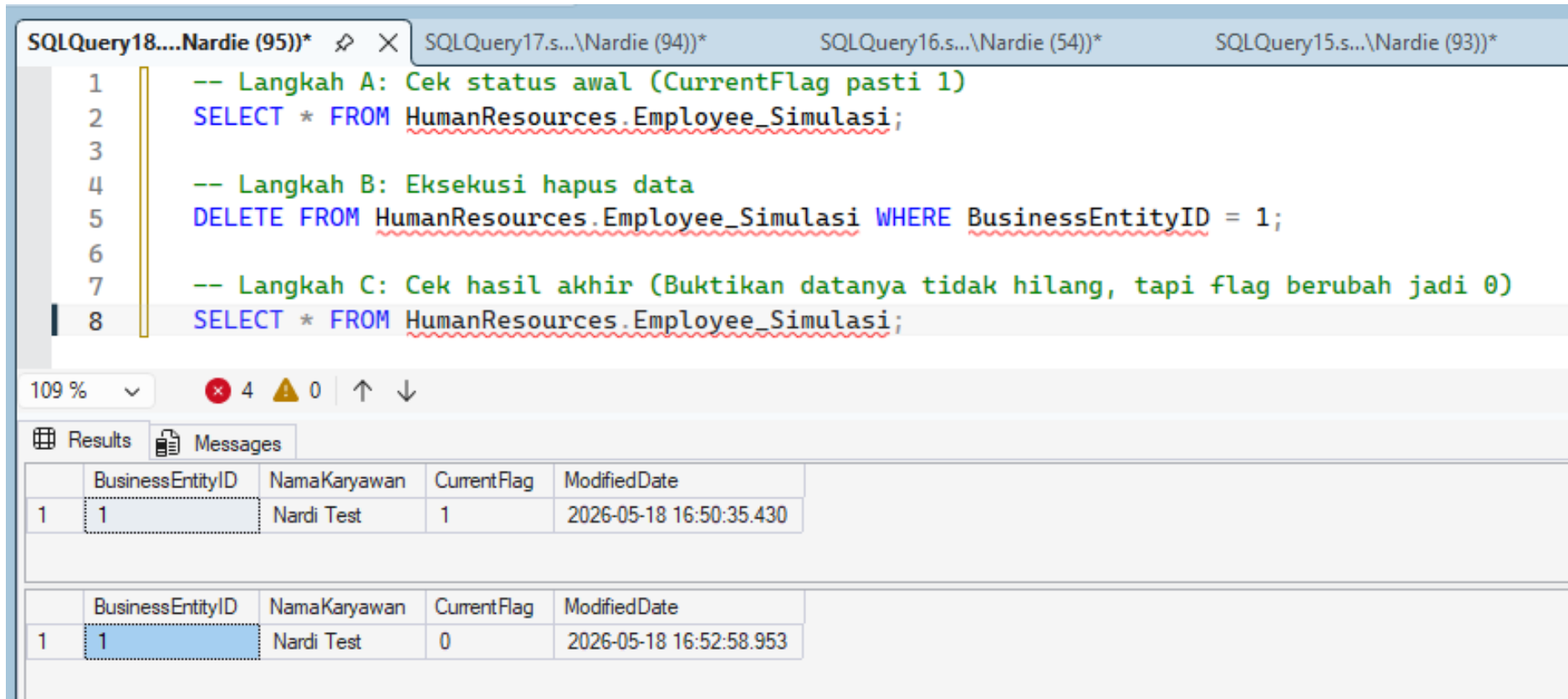
Messages

CREATE_TRIGGER - HumanResources.TR_EmployeeSimulasi_InsteadOfDelete

Completion time: 2026-05-18T16:51:45.9238683+07:00

Contoh Implementasi pada AdventureWorks2025

Jalankan Pengujian Akhir



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a SQL query with three steps: Step A (checking initial status), Step B (deleting data), and Step C (checking final status). The bottom pane shows the results of the query, which is a table with five columns: BusinessEntityID, NamaKaryawan, CurrentFlag, and ModifiedDate. The table contains two rows of data, both for BusinessEntityID 1. The first row shows CurrentFlag 1, and the second row shows CurrentFlag 0, indicating that the data was successfully deleted and the flag was updated.

```
1 -- Langkah A: Cek status awal (CurrentFlag pasti 1)
2 SELECT * FROM HumanResources.Employee_Simulasi;
3
4 -- Langkah B: Eksekusi hapus data
5 DELETE FROM HumanResources.Employee_Simulasi WHERE BusinessEntityID = 1;
6
7 -- Langkah C: Cek hasil akhir (Buktikan datanya tidak hilang, tapi flag berubah jadi 0)
8 SELECT * FROM HumanResources.Employee_Simulasi;
```

109 % 4 0

Results Messages

	BusinessEntityID	NamaKaryawan	CurrentFlag	ModifiedDate
1	1	Nardi Test	1	2026-05-18 16:50:35.430

	BusinessEntityID	NamaKaryawan	CurrentFlag	ModifiedDate
1	1	Nardi Test	0	2026-05-18 16:52:58.953

Contoh Implementasi pada AdventureWorks2025

Cara Memeriksa dan Mengelola Trigger

Melihat daftar trigger aktif di database:

```
SELECT name, is_instead_of_trigger, is_disabled  
FROM sys.triggers  
WHERE is_ms_shipped = 0;
```

Menonaktifkan Trigger (Disable):

```
DISABLE TRIGGER TR_Product_UpdatePrice ON Production.Product;
```

Mengaktifkan Kembali Trigger (Enable):

```
ENABLE TRIGGER TR_Product_UpdatePrice ON Production.Product;
```

Menghapus Trigger (Drop):

```
DROP TRIGGER TR_Product_UpdatePrice;
```

Tugas

1. Buat tabel stok barang (StokBarang) dan tabel penjualan (Penjualan)

Tabel Gudang / Stok

```
CREATE TABLE StokBarang (  
    NamaBarang VARCHAR(50) PRIMARY KEY,  
    Stok INT  
);
```

Tabel Kasir / Penjualan

```
CREATE TABLE Penjualan (  
    IDJual INT IDENTITY(1,1) PRIMARY KEY,  
    NamaBarang VARCHAR(50),  
    JumlahBeli INT  
);  
GO
```

2. Isi Stok Awal Barang dengan 100 bungkus Kopi

```
INSERT INTO StokBarang (NamaBarang, Stok) VALUES ('Kopi', 100);  
GO
```

Tugas

3. Buat Trigger Pengurang Stok Otomatis
4. Uji dengan anggapan ada pelanggan datang dan membeli 3 bungkus Kopi
5. Hasilnya kolom Stok untuk Kopi yang tadinya 100 akan langsung berubah otomatis menjadi 97!

Lakukan SS terhadap tahapan hasil dari pembuatan table, pembuatan trigger, uji trigger tsb!

TERIMA KASIH

TERIMA KASIH, MAHASISWA TEKNIK INFORMATIKA UNGGUL, ANDA SUDAH BELAJAR DENGAN LUAR BIASA

TERIMA KASIH, MAHASISWA TEKNIK INFORMATIKA UNGGUL, ANDA SUDAH BELAJAR DENGAN LUAR BIASA



Program Studi Teknik Informatika, Universitas Pamulang

